



1. Introduction

Libraries are essential hubs of knowledge and academic growth within any educational institution. However, managing these resources through traditional, manual methods often leads to severe inefficiencies, data redundancy, and a high likelihood of human error. The proposed Library Management System (LMS) is a comprehensive, digital database solution specifically designed to modernize and automate these critical library operations. The primary objectives of this project are to facilitate a seamless and highly efficient book borrowing and returning process, maintain accurate and real-time tracking of student activities and physical inventory, effectively manage employee daily tasks, and handle study room reservations systematically to completely eliminate scheduling conflicts and overlapping bookings. The scope of this project is extensive, encompassing the complete lifecycle of physical book inventory management, detailed transaction records for both borrowing and returning operations, dynamic study room bookings, and the secure management of both student and employee profiles. By transitioning to this centralized relational database, the system aims to drastically reduce paperwork, minimize administrative overhead, and provide a reliable, scalable foundation for future library enhancements.

2. Phase I Content: Requirements Analysis & Conceptual Design

The conceptual design phase served as the foundational blueprint for our database, where we meticulously gathered requirements and established the core data model. External Model: We defined three distinct user views to ensure the system meets the diverse needs of its users. First, the Student View allows users to search for books by title, author, or category, and view their active borrowing history. Second, the Employee View empowers library staff to manage physical inventory, register new students, and process daily borrowing and returning transactions. Third, the Library Manager View provides administrative oversight, enabling the manager to supervise employee shifts and generate comprehensive system reports. Business Rules: To maintain operational integrity, we established several strict business rules:

- Each physical book is uniquely identified by a globally unique Book_ID to prevent inventory confusion.
- A student is permitted to borrow multiple books concurrently, accommodating extensive research needs.
- Study Rooms require a valid, time-bound reservation linked directly to a Student_ID to ensure campus security and fair resource allocation.

Entities & Relationships: Through careful analysis, we identified the key entities that form the backbone of the system: Student, Employee, Manager, Book, Category, and Study_Room. We mapped complex relationships and cardinality constraints, ensuring that a Manager supervises



multiple Employees (1:N), and Categories systematically contain multiple Books (1:N). The conceptual Entity-Relationship (ER) diagram was successfully developed to visually represent these logical connections and data flows.

3. Phase I Feedback & Response

During our initial technical review, the IT Project Supervisor provided highly valuable and constructive feedback regarding the defined business rules and entity identifiers. Specifically, we were advised to refine our logical scope by removing two operational business rules that were more aligned with organizational policies rather than database constraints. Additionally, we were instructed to identify and define "Super Keys" for all entities to strengthen our understanding of uniqueness within the database. **Response:** We fully embraced and incorporated this feedback into our project. The specified business rules were successfully removed from the documentation to keep the database schema focused and technically sound. Furthermore, we conducted a thorough analysis of our attributes, identified the Super Keys for all entities, and officially integrated them into our subsequent logical design.

4. Phase II Content: Logical Design & Database Refinement

In this critical phase, the conceptual ERD was systematically transformed into a robust Relational Database Schema, ensuring data integrity and minimizing redundancy. Normalization (1NF-3NF): We applied rigorous normalization techniques using realistic sample data. First, we achieved First Normal Form (1NF) by eliminating all multi-valued attributes and repeating groups (e.g., breaking down multiple room bookings for a single student into atomic, individual rows). Next, we achieved Second Normal Form (2NF) by ensuring that all non-key attributes were fully functionally dependent on the entire primary key, removing any partial dependencies. Finally, we achieved Third Normal Form (3NF) by resolving transitive dependencies, which involved carefully separating the Employee processing data from the Booking_Room operational data. This process successfully eliminated insertion, update, and deletion anomalies. Design Refinements: A major challenge was resolving Many-to-Many (M:N) relationships inherent in library operations. We addressed this by introducing associative bridge entities. The Borrow and Return bridge entities were created to elegantly resolve the complex, time-based relationships between Students, Books, and Employees. Similarly, the Room_Booking associative entity was introduced to effectively break down the M:N relationship between Students and Study Rooms into manageable 1:N relationships. Final ERD: Following these refinements, all Primary Keys (PK) and Foreign Keys (FK)



were clearly defined and mapped. The result was a highly optimized, structurally sound Final ERD that perfectly represented the logical flow of the system.

5. Phase II Feedback & Response

Upon submitting the logical design, the detailed normalization steps, and the Final ERD to the **IT Department for review**, the structural integrity and logic of our database schema were carefully evaluated. The feedback received from the **technical team** was highly positive and encouraging. The **supervisor** confirmed that the Final ERD was exceptionally well-structured, logically accurate, and fully ready for physical deployment without needing further structural modifications. **Response:** With the official approval of the logical design and no corrective feedback required at this stage, I confidently proceeded directly to the physical implementation in Phase III, knowing the foundation was entirely solid.

6. Phase III Content: Physical Design & Implementation

The validated logical design was brought to life through a successful physical implementation using the MySQL Workbench environment. **Database & Tables Creation (DDL):** The physical cs311_library database was instantiated, containing 10 meticulously structured relational tables. We assigned precise and optimal data types to each attribute to ensure storage efficiency, such as using VARCHAR for alphanumeric identifiers, INT for numerical IDs, DATE and TIME for scheduling accuracy, and DECIMAL(10,2) for handling financial fines accurately. **Data Integrity Constraints:** We rigorously enforced data integrity at the structural level. We applied PRIMARY KEY constraints for unique identification and FOREIGN KEY constraints to strictly maintain referential integrity across all related tables. Additionally, we utilized UNIQUE constraints (for critical fields like Emails and ISBNs), NOT NULL to prevent missing mandatory data, and DEFAULT constraints to automate standard entries (e.g., defaulting a new room's status to 'Available'). **Business Rules via CHECK Constraints:** We took the implementation a step further by embedding our business logic directly into the database engine using CHECK constraints. For instance, we engineered a constraint ensuring that a room booking's Start_Time is strictly less than its End_Time. We also enforced a chronological rule ensuring that a Due_Date cannot logically precede a Borrow_Date, and mathematically guaranteed that financial Fine values cannot fall below zero. **Data Manipulation & Advanced Querying (DML & DQL):** The database was populated with a comprehensive set of integrated sample data to simulate a live library environment. We then rigorously evaluated the system's performance and



relational mapping using advanced SQL queries. This included utilizing multiple JOIN operations to trace specific student borrowings, deploying GROUP BY and HAVING clauses to calculate and filter employee fine collections, and designing a complex Nested Query (Subquery) to accurately identify students with active room bookings.

7. Phase III Feedback & Response

During the final technical evaluation, the physical database deployment and the complete MySQL scripts were thoroughly reviewed by the **IT supervisor**. The feedback highlighted the robust enforcement of data integrity, particularly commending the use of domain-specific CHECK constraints to handle business logic directly at the database level. Additionally, the advanced SQL queries were tested against the populated sample data, and the **technical team** verified their accuracy and efficiency in generating complex operational reports.

Response: I documented the successful query execution outputs and finalized the database schema based on this positive evaluation. With the backend architecture fully validated and proven to be operational, the database is now considered stable, secure, and fully prepared for potential future integration with a front-end application or API services.

8. Conclusion

The Library Management System project represents a highly successful transition from a theoretical conceptual idea into a fully functional, deployable relational database. Through a systematic approach—starting with in-depth requirements analysis, progressing through rigorous mathematical normalization, and culminating in a precise physical implementation using MySQL—I developed a robust system that guarantees high data integrity, structural consistency, and highly efficient information retrieval. The iterative feedback loop with **my training supervisor at Taibah University** was instrumental in streamlining the logical schema and elevating the professional quality of my work.

Working on this project **during my cooperative IT training** solidified my understanding of core database concepts, particularly the importance of bridging entities and the power of embedding business logic directly into SQL constraints. Future enhancements for this system could include the implementation of automated SQL Triggers for real-time fine calculations, the integration of Stored Procedures for daily reporting, and ultimately developing a modern Graphical User Interface (GUI) to allow end-users to interact seamlessly with the powerful backend database I have constructed.



9. Appendices

- **Appendix A: Full MySQL Script (DDL & DML).**

```
-- =====  
  
-- PART 1: DDL (DATA DEFINITION LANGUAGE)  
  
-- CREATING DATABASE, TABLES, AND CONSTRAINTS  
  
-- =====  
  
CREATE DATABASE IF NOT EXISTS cs311_library;  
  
USE cs311_library;  
  
  
CREATE TABLE Category (  
    Category_ID INT PRIMARY KEY,  
    Category_Name VARCHAR(100) NOT NULL UNIQUE  
);  
  
  
CREATE TABLE Manager (  
    Manager_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE NOT NULL,  
    Office VARCHAR(50) DEFAULT 'Main Office'  
);
```



```
CREATE TABLE Study_Room (  
    Room_ID VARCHAR(50) PRIMARY KEY,  
    Status VARCHAR(50) NOT NULL DEFAULT 'Available' CHECK (Status IN ('Available',  
'Booked')),  
    Capacity INT NOT NULL CHECK (Capacity >= 2 AND Capacity <= 10)  
);
```

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE NOT NULL  
);
```

```
CREATE TABLE Employee (  
    Employee_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE NOT NULL,  
    Position VARCHAR(50) NOT NULL DEFAULT 'Staff',  
    Manager_ID INT,  
    FOREIGN KEY (Manager_ID) REFERENCES Manager(Manager_ID)  
);
```

```
CREATE TABLE Book (  
    
```



```
Book_ID VARCHAR(50) PRIMARY KEY,  
Title VARCHAR(200) NOT NULL,  
Author VARCHAR(100) NOT NULL,  
ISBN VARCHAR(50) UNIQUE NOT NULL,  
Publisher VARCHAR(100),  
Year INT CHECK (Year >= 1900 AND Year <= 2026),  
Status VARCHAR(50) DEFAULT 'Available' CHECK (Status IN ('Available', 'Borrowed',  
'Lost')),  
Category_ID INT,  
FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)  
);
```

```
CREATE TABLE Room_Booking (  
Booking_ID VARCHAR(50) PRIMARY KEY,  
Student_ID INT NOT NULL,  
Room_ID VARCHAR(50) NOT NULL,  
Booking_Date DATE NOT NULL,  
Start_Time TIME NOT NULL,  
End_Time TIME NOT NULL,  
Employee_ID INT NOT NULL,  
FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),  
FOREIGN KEY (Room_ID) REFERENCES Study_Room(Room_ID),  
FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),  
CHECK (Start_Time < End_Time)
```



);

```
CREATE TABLE Borrow (  
    Borrow_ID VARCHAR(50) PRIMARY KEY,  
    Borrow_Date DATE NOT NULL,  
    Due_Date DATE NOT NULL,  
    Book_ID VARCHAR(50) NOT NULL,  
    Student_ID INT NOT NULL,  
    FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),  
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),  
    CHECK (Due_Date >= Borrow_Date)  
);
```

```
CREATE TABLE Book_Update (  
    Update_ID VARCHAR(50) PRIMARY KEY,  
    Book_ID VARCHAR(50) NOT NULL,  
    Employee_ID INT NOT NULL,  
    Update_Date DATE NOT NULL,  
    FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),  
    FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID)
```




);

CREATE TABLE `Return` (

Return_ID VARCHAR(50) PRIMARY KEY,

Return_Date DATE NOT NULL,

Borrow_ID VARCHAR(50) NOT NULL,

Employee_ID INT NOT NULL,

Fine DECIMAL(10,2) DEFAULT 0.00 CHECK (Fine >= 0),

Book_ID VARCHAR(50) NOT NULL,

FOREIGN KEY (Borrow_ID) REFERENCES Borrow(Borrow_ID),

FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),

FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID)

);



-- PART 2: DML (DATA MANIPULATION LANGUAGE)

-- INSERTING SAMPLE DATA (10 ROWS PER TABLE)

-- =====

USE cs311_library;

SET FOREIGN_KEY_CHECKS = 0;

SET SQL_SAFE_UPDATES = 0;

-- 1. Category Table

DELETE FROM Category;

INSERT INTO Category (Category_ID, Category_Name) VALUES

(1, 'Database Systems'),

(2, 'Software Engineering'),

(3, 'Operating Systems'),

(4, 'Computer Networks'),

(5, 'Cybersecurity'),

(6, 'Artificial Intelligence'),

(7, 'Embedded Systems'),

(8, 'Data Science'),



(9, 'Web Development'),

(10, 'Algorithms & Data Structures');

-- 2. Manager Table (10 Rows)

DELETE FROM Manager;

INSERT INTO Manager (Manager_ID, Name, Email, Office) VALUES

(101, 'Dr. Kajal Nusratullah', 'kajal@yic.edu.sa', 'Main Office A-10'),

(102, 'Dr. Ahmed Al-Ahmadi', 'a.ahmadi@yic.edu.sa', 'Admin Office B-05'),

(103, 'Dr. Sarah Smith', 'ssmith@yic.edu.sa', 'Office A-11'),

(104, 'Dr. Ali Hassan', 'ahassan@yic.edu.sa', 'Office B-06'),

(105, 'Dr. Mona Zaki', 'mzaki@yic.edu.sa', 'Office A-12'),

(106, 'Dr. Yasser Omar', 'yomar@yic.edu.sa', 'Office B-07'),

(107, 'Dr. Huda Khaled', 'hkhaled@yic.edu.sa', 'Office A-13'),

(108, 'Dr. Sami Saeed', 'ssaeed@yic.edu.sa', 'Office B-08'),

(109, 'Dr. Nada Tariq', 'ntariq@yic.edu.sa', 'Office A-14'),

(110, 'Dr. Fahad Salem', 'fsalem@yic.edu.sa', 'Office B-09');

-- 3. Student Table (10 Rows)



DELETE FROM Student;

INSERT INTO Student (Student_ID, Name, Email) VALUES

(451500543, 'Layan Aljohani', 'layan@yic.edu.sa'),
(451500603, 'Alhanouf Aljohani', 'alhanouf@yic.edu.sa'),
(451500569, 'Wadeef Alalwani', 'wadeef@yic.edu.sa'),
(451500101, 'Sara Al-Ghamdi', 'sara.g@yic.edu.sa'),
(451500102, 'Noura Al-Otaibi', 'noura.o@yic.edu.sa'),
(451500103, 'Reem Al-Zahrani', 'reem.z@yic.edu.sa'),
(451500104, 'Mona Al-Harbi', 'mona.h@yic.edu.sa'),
(451500105, 'Amal Al-Mutairi', 'amal.m@yic.edu.sa'),
(451500106, 'Hana Al-Shehri', 'hana.s@yic.edu.sa'),
(451500107, 'Shahad Al-Kahlani', 'shahad.k@yic.edu.sa');

-- 4. Employee Table (10 Rows)

DELETE FROM Employee;

INSERT INTO Employee (Employee_ID, Name, Email, Position, Manager_ID) VALUES

(44515, 'Mona', 'mona@lib.com', 'Librarian', 101),
(44315, 'Rawan', 'rawan@lib.com', 'Assistant', 101),
(44316, 'Huda', 'huda@lib.com', 'Cataloger', 101),
(44317, 'Fatimah', 'fatimah@lib.com', 'Circulation Clerk', 102),
(44318, 'Nouf', 'nouf@lib.com', 'Assistant Librarian', 102),



(44319, 'Reem', 'reem@lib.com', 'IT Support', 103),
(44320, 'Amal', 'amal@lib.com', 'Archivist', 104),
(44321, 'Laila', 'laila@lib.com', 'Librarian', 105),
(44322, 'Sara', 'sara.emp@lib.com', 'Assistant', 106),
(44323, 'Nawal', 'nawal@lib.com', 'Clerk', 107);

-- 5. Study_Room Table (10 Rows)

DELETE FROM Study_Room;

INSERT INTO Study_Room (Room_ID, Status, Capacity) VALUES

('R101', 'Available', 5),

('R102', 'Available', 8),

('R103', 'Booked', 4),

('R104', 'Available', 6),

('R105', 'Booked', 4),

('R106', 'Available', 10),

('R107', 'Booked', 4),

('R108', 'Available', 6),

('R109', 'Available', 5),

('R110', 'Booked', 2);



-- 6. Book Table (10 Rows)

DELETE FROM Book;

INSERT INTO Book (Book_ID, Title, Author, ISBN, Publisher, Year, Status, Category_ID)
VALUES

('BK_1', 'Database System Concepts', 'Abraham Silberschatz', '978-1111', 'McGraw-Hill', 2024,
'Available', 1),

('BK_2', 'Software Engineering Principles', 'Ian Sommerville', '978-2222', 'Pearson', 2025,
'Available', 2),

('BK_3', 'Operating System Concepts', 'Peter Galvin', '978-3333', 'Wiley', 2024, 'Borrowed', 3),

('BK_4', 'Computer Networking: Top-Down Approach', 'James Kurose', '978-4444', 'Pearson',
2023, 'Available', 4),

('BK_5', 'Computer Security Principles', 'William Stallings', '978-5555', 'Pearson', 2024,
'Borrowed', 5),

('BK_6', 'Artificial Intelligence: A Modern Approach', 'Stuart Russell', '978-6666', 'Pearson',
2023, 'Available', 6),

('BK_7', 'Embedded Systems Architecture', 'Tammie Noergaard', '978-7777', 'Elsevier', 2022,
'Available', 7),

('BK_8', 'Python for Data Analysis', 'Wes McKinney', '978-8888', 'O Reilly', 2023, 'Borrowed',
8),

('BK_9', 'Learning Web Design', 'Jennifer Robbins', '978-9999', 'O Reilly', 2022, 'Available', 9),

('BK_10', 'Introduction to Algorithms', 'Thomas Cormen', '978-1010', 'MIT Press', 2024,
'Available', 10);

-- 7. Borrow Table (10 Rows)

DELETE FROM Borrow;



```
INSERT INTO Borrow (Borrow_ID, Borrow_Date, Due_Date,  
Book_ID, Student_ID) VALUES
```

```
('BR_1', '2026-07-20', '2026-07-27', 'BK_1', 451500543),  
('BR_2', '2026-07-21', '2026-07-28', 'BK_3', 451500603),  
('BR_3', '2026-07-10', '2026-07-17', 'BK_5', 451500569),  
('BR_4', '2026-07-15', '2026-07-22', 'BK_8', 451500101),  
('BR_5', '2026-07-25', '2026-08-01', 'BK_2', 451500102),  
('BR_6', '2026-07-26', '2026-08-02', 'BK_4', 451500103),  
('BR_7', '2026-07-27', '2026-08-03', 'BK_6', 451500104),  
('BR_8', '2026-07-28', '2026-08-04', 'BK_7', 451500105),  
('BR_9', '2026-07-29', '2026-08-05', 'BK_9', 451500106),  
('BR_10', '2026-07-30', '2026-08-06', 'BK_10', 451500107);
```

-- 8. Return Table (10 Rows)

```
DELETE FROM `Return`;
```

```
INSERT INTO `Return` (Return_ID, Return_Date, Borrow_ID, Employee_ID, Fine, Book_ID)  
VALUES
```

```
('RET_1', '2026-07-28', 'BR_1', 44515, 15.50, 'BK_1'),  
('RET_2', '2026-07-20', 'BR_3', 44315, 25.00, 'BK_5'),  
('RET_3', '2026-07-22', 'BR_4', 44515, 0.00, 'BK_8'),  
('RET_4', '2026-07-29', 'BR_2', 44316, 5.00, 'BK_3'),  
('RET_5', '2026-08-01', 'BR_5', 44317, 0.00, 'BK_2'),
```



```
('RET_6', '2026-08-03', 'BR_6', 44318, 10.00, 'BK_4'),  
( 'RET_7', '2026-08-05', 'BR_7', 44319, 0.00, 'BK_6'),  
( 'RET_8', '2026-08-05', 'BR_8', 44320, 15.00, 'BK_7'),  
( 'RET_9', '2026-08-06', 'BR_9', 44321, 0.00, 'BK_9'),  
( 'RET_10', '2026-08-07', 'BR_10', 44322, 0.00, 'BK_10');
```

-- 9. Room_Booking Table (10 Rows)

```
DELETE FROM Room_Booking;
```

```
INSERT INTO Room_Booking (Booking_ID, Student_ID, Room_ID, Booking_Date,  
Start_Time, End_Time, Employee_ID) VALUES
```

```
('RB_1', 451500543, 'R101', '2026-07-22', '10:00:00', '12:00:00', 44515),  
( 'RB_2', 451500603, 'R102', '2026-07-25', '13:00:00', '15:00:00', 44515),  
( 'RB_3', 451500569, 'R101', '2026-07-23', '09:00:00', '11:00:00', 44315),  
( 'RB_4', 451500101, 'R103', '2026-07-26', '14:00:00', '16:00:00', 44316),  
( 'RB_5', 451500102, 'R104', '2026-07-27', '11:00:00', '13:00:00', 44317),  
( 'RB_6', 451500103, 'R106', '2026-07-28', '08:00:00', '10:00:00', 44318),  
( 'RB_7', 451500104, 'R107', '2026-07-29', '09:00:00', '11:00:00', 44319),  
( 'RB_8', 451500105, 'R108', '2026-07-30', '12:00:00', '14:00:00', 44320),  
( 'RB_9', 451500106, 'R109', '2026-07-31', '13:00:00', '15:00:00', 44321),  
( 'RB_10', 451500107, 'R101', '2026-08-01', '10:00:00', '12:00:00', 44322);
```




-- 10. Book_Update Table (10 Rows)

DELETE FROM Book_Update;

INSERT INTO Book_Update (Update_ID, Book_ID, Employee_ID, Update_Date) VALUES

('UP_1', 'BK_1', 44515, '2026-07-01'),

('UP_2', 'BK_2', 44315, '2026-07-15'),

('UP_3', 'BK_5', 44316, '2026-07-18'),

('UP_4', 'BK_3', 44317, '2026-07-19'),

('UP_5', 'BK_4', 44318, '2026-07-20'),

('UP_6', 'BK_6', 44319, '2026-07-21'),

('UP_7', 'BK_7', 44320, '2026-07-22'),

('UP_8', 'BK_8', 44321, '2026-07-23'),

('UP_9', 'BK_9', 44322, '2026-07-24'),

('UP_10', 'BK_10', 44323, '2026-07-25');

SET FOREIGN_KEY_CHECKS = 1;



• Appendix B: Screenshots of MySQL Workbench Table Creation and Constraints implementation.

```

1  -- PART 1: DDL (DATA DEFINITION LANGUAGE)
2  -- CREATING DATABASE, TABLES, AND CONSTRAINTS
3  -- =====
4
5  CREATE DATABASE IF NOT EXISTS cs311_library;
6  USE cs311_library;
7
8  CREATE TABLE Category (
9      Category_ID INT PRIMARY KEY,
10     Category_Name VARCHAR(100) NOT NULL UNIQUE
11 );
12
13 CREATE TABLE Manager (
14     Manager_ID INT PRIMARY KEY,
15     Name VARCHAR(100) NOT NULL,
16     Email VARCHAR(100) UNIQUE NOT NULL,
17     Office VARCHAR(50) DEFAULT 'Main Office'
18 );
19
20
21 CREATE TABLE Study_Room (
22     Room_ID VARCHAR(50) PRIMARY KEY,
23     Status VARCHAR(50) NOT NULL DEFAULT 'Available' CHECK (Status IN ('Available', 'Booked')),
24     Capacity INT NOT NULL CHECK (Capacity >= 2 AND Capacity <= 10)
25 );
26
27 CREATE TABLE Student (
28     Student_ID INT PRIMARY KEY,
29     Name VARCHAR(100) NOT NULL,
30     Email VARCHAR(100) UNIQUE NOT NULL
31 );
32
33 CREATE TABLE Employee (
34     Employee_ID INT PRIMARY KEY,
35     Name VARCHAR(100) NOT NULL,
36     Email VARCHAR(100) UNIQUE NOT NULL,
37     Position VARCHAR(50) NOT NULL DEFAULT 'Staff',
38     Manager_ID INT,
39     FOREIGN KEY (Manager_ID) REFERENCES Manager(Manager_ID)
40 );
41
42 CREATE TABLE Book (
43     Book_ID VARCHAR(50) PRIMARY KEY,

```

```

44     Book_ID VARCHAR(50) PRIMARY KEY,
45     Title VARCHAR(200) NOT NULL,
46     Author VARCHAR(100) NOT NULL,
47     ISBN VARCHAR(50) UNIQUE NOT NULL,
48     Publisher VARCHAR(100),
49     Year INT CHECK (Year >= 1900 AND Year <= 2026),
50     Status VARCHAR(50) DEFAULT 'Available' CHECK (Status IN ('Available', 'Borrowed', 'Lost')),
51     Category_ID INT,
52     FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)
53 );
54
55 CREATE TABLE Room_Booking (
56     Booking_ID VARCHAR(50) PRIMARY KEY,
57     Student_ID INT NOT NULL,
58     Room_ID VARCHAR(50) NOT NULL,
59     Booking_Date DATE NOT NULL,
60     Start_Time TIME NOT NULL,
61     End_Time TIME NOT NULL,
62     Employee_ID INT NOT NULL,
63     FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
64     FOREIGN KEY (Room_ID) REFERENCES Study_Room(Room_ID),
65     FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),
66     CHECK (Start_Time < End_Time)
67 );
68
69 CREATE TABLE Borrow (
70     Borrow_ID VARCHAR(50) PRIMARY KEY,
71     Borrow_Date DATE NOT NULL,
72     Due_Date DATE NOT NULL,
73     Book_ID VARCHAR(50) NOT NULL,
74     Student_ID INT NOT NULL,
75     FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),
76     FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
77 );

```



SCHEMAS

Filter objects

cs311_library

- Tables
 - Book
 - Book_Update
 - Borrow
 - Category
 - Employee
 - Manager
 - Return
 - Room_Booking
 - Student
 - Study_Room
- Views
- Stored Procedures
- Functions

Object Info

Session

Schema: cs311_library

```

65 );
66
67 CREATE TABLE Borrow (
68     Borrow_ID VARCHAR(50) PRIMARY KEY,
69     Borrow_Date DATE NOT NULL,
70     Due_Date DATE NOT NULL,
71     Book_ID VARCHAR(50) NOT NULL,
72     Student_ID INT NOT NULL,
73     FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),
74     FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
75
76     CHECK (Due_Date >= Borrow_Date)
77 );
78
79 CREATE TABLE Book_Update (
80     Update_ID VARCHAR(50) PRIMARY KEY,
81     Book_ID VARCHAR(50) NOT NULL,
82     Employee_ID INT NOT NULL,
83     Update_Date DATE NOT NULL,
84     FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),
85     FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID)
86 );
87
88 CREATE TABLE 'Return' (
89     Return_ID VARCHAR(50) PRIMARY KEY,
90     Return_Date DATE NOT NULL,
91     Borrow_ID VARCHAR(50) NOT NULL,
92     Employee_ID INT NOT NULL,
93     Fine DECIMAL(10,2) DEFAULT 0.00 CHECK (Fine >= 0),
94     Book_ID VARCHAR(50) NOT NULL,
95     FOREIGN KEY (Borrow_ID) REFERENCES Borrow(Borrow_ID),
96     FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),
97     FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID)
98 );
    
```

100% 1:87



- **Appendix C: Screenshots of successfully executed SQL Queries (SELECT, JOIN, GROUP BY, Nested) and their respective Result Grids.**

```

1 • SELECT s.Name AS Student_Name, b.Title AS Book_Title, br.Due_Date
2   FROM Student s
3  JOIN Borrow br ON s.Student_ID = br.Student_ID
4  JOIN Book b ON br.Book_ID = b.Book_ID;

```

160% 39.4

Result Grid Filter Rows: Search Export:

Student_Name	Book_Title	Due_Date
Layan Aljohani	Database System Concepts	2026-07-27
Alhanouf Aljohani	Operating System Concepts	2026-07-28
Wadeef Alalwani	Computer Security Principles	2026-07-17
Sara Al-Ghamdi	Python for Data Analysis	2026-07-22
Noura Al-Otaibi	Software Engineering Principles	2026-08-01

Result 2

Action Output

	Time	Action	Response
1	12:10:24	SELECT s.Name AS Student_Name, b.Title AS Book_Title, br.Due_Date FROM Student s JOIN Borrow b...	5 row(s) returned



```

1 • SELECT e.Name AS Employee_Name, SUM(r.Fine) AS Total_Fines_Collected
2   FROM Employee e
3  JOIN `Return` r ON e.Employee_ID = r.Employee_ID
4  GROUP BY e.Name
5  HAVING Total_Fines_Collected > 0;
6

```

Result Grid

Employee_Name	Total_Fines_Collect...
Mona	15.50
Rawan	25.00

Result 1

Action Output

	Time	Action	Response
✓ 1	12:10:24	SELECT s.Name AS Student_Name, b.Title AS Book_Title, br.Due_Date FROM Student s JOIN Borrow b...	5 row(s) returned
✓ 2	12:12:32	SELECT e.Name AS Employee_Name, SUM(r.Fine) AS Total_Fines_Collected FROM Employee e JOIN `...	2 row(s) returned

```

1 • SELECT Name, Email
2   FROM Student
3  WHERE Student_ID IN (
4      SELECT Student_ID
5      FROM Room_Booking
6  );
7

```

Result Grid

Name	Email
Sara Al-Ghamdi	sara.g@yic.edu.sa
Noura Al-Otaibi	noura.o@yic.edu.sa
Layan Aljohani	layan@yic.edu.sa
Wadeef Alalwani	wadeef@yic.edu.sa
Alhanouf Aljohani	alhanouf@yic.edu.sa

Student 5

Action Output

	Time	Action	Response
✓ 1	12:14:29	SELECT Name, Email FROM Student WHERE Student_ID IN (SELECT Student_ID FROM Room_B...	5 row(s) returned



• Appendix D: Sample Data - Table Contents

Limit to 1000 rows

```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

Result Grid

Student_ID	Name	Email
451500101	Sara Al-Ghamdi	sara.g@yic.edu.sa
451500102	Noura Al-Otaibi	noura.o@yic.edu.sa
451500103	Reem Al-Zahrani	reem.z@yic.edu.sa
451500104	Mona Al-Harbi	mona.h@yic.edu.sa
451500105	Amal Al-Mutairi	amal.m@yic.edu.sa
451500106	Hana Al-Shehri	hana.s@yic.edu.sa
451500107	Shahad Al-Kahlani	shahad.k@yic.edu.sa
451500543	Layan Aljohani	layan@yic.edu.sa
451500569	Wadeef Alalwani	wadeef@yic.edu.sa
451500603	Alhanouf Aljohani	alhanouf@yic.edu.sa
NULL	NULL	NULL

Student 21 Book 22 Employee 23 Manager 24 Category 25 Study_Room 26 Borrow 27 Return 28 >> Apply Revert

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 4	12:35:26	SELECT * FROM Manager LIMIT 0, 1000	10 row(s) returned	0.00056 sec / 0.000...
✓ 5	12:35:26	SELECT * FROM Category LIMIT 0, 1000	10 row(s) returned	0.00056 sec / 0.000...
✓ 6	12:35:26	SELECT * FROM Study_Room LIMIT 0, 1000	10 row(s) returned	0.00046 sec / 0.000...
✓ 7	12:35:26	SELECT * FROM Borrow LIMIT 0, 1000	10 row(s) returned	0.00047 sec / 0.0000...
✓ 8	12:35:26	SELECT * FROM `Return` LIMIT 0, 1000	10 row(s) returned	0.00032 sec / 0.0000...
✓ 9	12:35:26	SELECT * FROM Room_Booking LIMIT 0, 1000	10 row(s) returned	0.00032 sec / 0.0000...
✓ 10	12:35:26	SELECT * FROM Book_Update LIMIT 0, 1000	10 row(s) returned	0.00027 sec / 0.0000...



Limit to 1000 rows

```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

Result Grid Filter Rows: Search Edit: Export/Import:

Book_ID	Title	Author	ISBN	Publisher	Year	Status	Category_ID
BK_1	Database System Concepts	Abraham Silberschatz	978-1111	McGraw-Hill	2024	Available	1
BK_10	Introduction to Algorithms	Thomas Cormen	978-1010	MIT Press	2024	Available	10
BK_2	Software Engineering Principles	Ian Sommerville	978-2222	Pearson	2025	Available	2
BK_3	Operating System Concepts	Peter Galvin	978-3333	Wiley	2024	Borrowed	3
BK_4	Computer Networking: Top-Down Approach	James Kurose	978-4444	Pearson	2023	Available	4
BK_5	Computer Security Principles	William Stallings	978-5555	Pearson	2024	Borrowed	5
BK_6	Artificial Intelligence: A Modern Approach	Stuart Russell	978-6666	Pearson	2023	Available	6
BK_7	Embedded Systems Architecture	Tammie Noergaard	978-7777	Elsevier	2022	Available	7
BK_8	Python for Data Analysis	Wes McKinney	978-8888	O Reilly	2023	Borrowed	8
BK_9	Learning Web Design	Jennifer Robbins	978-9999	O Reilly	2022	Available	9
HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL

Student 21 Book 22 Employee 23 Manager 24 Category 25 Study_Room 26 Borrow 27 Return 28 >>

```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

Result Grid Filter Rows: Search Edit: Export/Import:

Employee_ID	Name	Email	Position	Manager_ID
44315	Rawan	rawan@lib.com	Assistant	101
44316	Huda	huda@lib.com	Cataloger	101
44317	Fatimah	fatimah@lib.com	Circulation Clerk	102
44318	Nouf	nouf@lib.com	Assistant Librarian	102
44319	Reem	reem@lib.com	IT Support	103
44320	Amal	amal@lib.com	Archivist	104
44321	Laila	laila@lib.com	Librarian	105
44322	Sara	sara.emp@lib.com	Assistant	106
44323	Nawal	nawal@lib.com	Clerk	107
44515	Mona	mona@lib.com	Librarian	101
HULL	HULL	HULL	HULL	HULL

Student 21 Book 22 Employee 23 Manager 24 Category 25 Study_Room 26 Borrow 27 Return 28 >>



1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

Result Grid Filter Rows: Search Edit: Export/Import:

Manager_ID	Name	Email	Office
101	Dr. Kajal Nusratullah	kajal@yic.edu.sa	Main Office A-10
102	Dr. Ahmed Al-Ahmadi	a.ahmadi@yic.edu.sa	Admin Office B-05
103	Dr. Sarah Smith	ssmith@yic.edu.sa	Office A-11
104	Dr. Ali Hassan	ahassan@yic.edu.sa	Office B-06
105	Dr. Mona Zaki	mzaki@yic.edu.sa	Office A-12
106	Dr. Yasser Omar	yomar@yic.edu.sa	Office B-07
107	Dr. Huda Khaled	hkhaled@yic.edu.sa	Office A-13
108	Dr. Sami Saeed	ssaeed@yic.edu.sa	Office B-08
109	Dr. Nada Tariq	ntariq@yic.edu.sa	Office A-14
110	Dr. Fahad Salem	fsalem@yic.edu.sa	Office B-09
NULL	NULL	NULL	NULL

Student 21 Book 22 Employee 23 Manager 24 x Category 25 Study_Room 26 Borrow 27 Return 28 >>

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

Result Grid Filter Rows: Search Edit: Export/Import:

Category_ID	Category_Name
10	Algorithms & Data Structures
6	Artificial Intelligence
4	Computer Networks
5	Cybersecurity
8	Data Science
1	Database Systems
7	Embedded Systems
3	Operating Systems
2	Software Engineering
9	Web Development
NULL	NULL

Student 21 Book 22 Employee 23 Manager 24 Category 25 Study_Room 26 Borrow 27 Return 28 >>



```
1 • SELECT * FROM Study_Room;
2
```

250% 1:4

Result Grid



Filter Rows:

Search

Edit:



Export/Import:



Room_ID	Status	Capacity
R101	Available	5
R102	Available	8
R103	Booked	4
R104	Available	6
R105	Booked	4
R106	Available	10
R107	Booked	4
R108	Available	6
R109	Available	5
R110	Booked	2
NULL	NULL	NULL

```
1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11
```

1:12

Result Grid



Filter Rows:

Search

Edit:



Export/Import:



Borrow_ID	Borrow_Date	Due_Date	Book_ID	Student_ID
BR_1	2026-07-20	2026-07-27	BK_1	451500543
BR_10	2026-07-30	2026-08-06	BK_10	451500107
BR_2	2026-07-21	2026-07-28	BK_3	451500603
BR_3	2026-07-10	2026-07-17	BK_5	451500569
BR_4	2026-07-15	2026-07-22	BK_8	451500101
BR_5	2026-07-25	2026-08-01	BK_2	451500102
BR_6	2026-07-26	2026-08-02	BK_4	451500103
BR_7	2026-07-27	2026-08-03	BK_6	451500104
BR_8	2026-07-28	2026-08-04	BK_7	451500105
BR_9	2026-07-29	2026-08-05	BK_9	451500106
NULL	NULL	NULL	NULL	NULL

Student 21

Book 22

Employee 23

Manager 24

Category 25

Study_Room 26

Borrow 27

Return 28

>>



```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

1:12

Result Grid Filter Rows: Search Edit: Export/Import:

Return_ID	Return_Date	Borrow_ID	Employee_ID	Fine	Book_ID
RET_1	2026-07-28	BR_1	44515	15.50	BK_1
RET_10	2026-08-07	BR_10	44322	0.00	BK_10
RET_2	2026-07-20	BR_3	44315	25.00	BK_5
RET_3	2026-07-22	BR_4	44515	0.00	BK_8
RET_4	2026-07-29	BR_2	44316	5.00	BK_3
RET_5	2026-08-01	BR_5	44317	0.00	BK_2
RET_6	2026-08-03	BR_6	44318	10.00	BK_4
RET_7	2026-08-05	BR_7	44319	0.00	BK_6
RET_8	2026-08-05	BR_8	44320	15.00	BK_7
RET_9	2026-08-06	BR_9	44321	0.00	BK_9
NULL	NULL	NULL	NULL	NULL	NULL

Student 21

Book 22

Employee 23

Manager 24

Category 25

Study_Room 26

Borrow 27

Return 28

>>

```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

1:12

Result Grid Filter Rows: Search Edit: Export/Import:

Booking_ID	Student_ID	Room_ID	Booking_Date	Start_Time	End_Time	Employee_ID
RB_1	451500543	R101	2026-07-22	10:00:00	12:00:00	44515
RB_10	451500107	R101	2026-08-01	10:00:00	12:00:00	44322
RB_2	451500603	R102	2026-07-25	13:00:00	15:00:00	44515
RB_3	451500569	R101	2026-07-23	09:00:00	11:00:00	44315
RB_4	451500101	R103	2026-07-26	14:00:00	16:00:00	44316
RB_5	451500102	R104	2026-07-27	11:00:00	13:00:00	44317
RB_6	451500103	R106	2026-07-28	08:00:00	10:00:00	44318
RB_7	451500104	R107	2026-07-29	09:00:00	11:00:00	44319
RB_8	451500105	R108	2026-07-30	12:00:00	14:00:00	44320
RB_9	451500106	R109	2026-07-31	13:00:00	15:00:00	44321
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Book 22

Employee 23

Manager 24

Category 25

Study_Room 26

Borrow 27

Return 28

Room_Booking 29

>>



```

1 • SELECT * FROM Student;
2 • SELECT * FROM Book;
3 • SELECT * FROM Employee;
4 • SELECT * FROM Manager;
5 • SELECT * FROM Category;
6 • SELECT * FROM Study_Room;
7 • SELECT * FROM Borrow;
8 • SELECT * FROM `Return`;
9 • SELECT * FROM Room_Booking;
10 • SELECT * FROM Book_Update;
11

```

Result Grid

Update_ID	Book_ID	Employee_ID	Update_Date
UP_1	BK_1	44515	2026-07-01
UP_10	BK_10	44323	2026-07-25
UP_2	BK_2	44315	2026-07-15
UP_3	BK_5	44316	2026-07-18
UP_4	BK_3	44317	2026-07-19
UP_5	BK_4	44318	2026-07-20
UP_6	BK_6	44319	2026-07-21
UP_7	BK_7	44320	2026-07-22
UP_8	BK_8	44321	2026-07-23
UP_9	BK_9	44322	2026-07-24
NULL	NULL	NULL	NULL

Employee 23 Manager 24 Category 25 Study_Room 26 Borrow 27 Return 28 Room_Booking 29 Book_Update 30 >>

Figure: Constraint Enforcement Test – Rejection of Invalid Room Capacity (Capacity < 2)

SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15* SQL File 16* >> Context...

Limit to 1000 rows

```

1 • INSERT INTO Study_Room (Room_ID, Status, Capacity) VALUES ('R999', 'Available', 1);

```

100% 84:1

Action Output

	Time	Action	Response	Duration / Fetch Time
✖ 1	15:06:44	DROP TABLE IF EXISTS Study_Room	Error Code: 3730. Cannot drop table 'study_room' ref...	0.00066 sec
✔ 2	15:07:18	SET FOREIGN_KEY_CHECKS = 0	0 row(s) affected	0.0081 sec
✔ 3	15:07:18	DROP TABLE IF EXISTS Study_Room	0 row(s) affected	0.070 sec
✔ 4	15:07:18	CREATE TABLE Study_Room (Room_ID VARCHAR(50) PRIMARY KEY, Status VAR...	0 row(s) affected	0.025 sec
✔ 5	15:07:18	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.00022 sec
✖ 6	15:07:43	INSERT INTO Study_Room (Room_ID, Status, Capacity) VALUES ('R999', 'Available', 1)	Error Code: 3819. Check constraint 'study_room_chk_...	0.0048 sec